

Rapport Détaillé du Backend et du Frontend

Plateforme web de mise en relation entre clients et freelances

Table des matières

1	Explication détaillée du Backend	3
1.1	Rôle du Backend	3
1.2	Technologies utilisées dans le Backend	3
1.3	Organisation du Backend	3
1.3.1	Dossiers importants	3
1.4	Authentification et gestion des utilisateurs	4
1.4.1	Inscription	4
1.4.2	Connexion	4
1.4.3	Profil	4
1.4.4	Déconnexion	5
1.5	Réinitialisation du mot de passe	5
1.5.1	Demande de réinitialisation	5
1.5.2	Mail envoyé	5
1.5.3	Définition du nouveau mot de passe	5
1.6	Le modèle Utilisateur	6
1.6.1	Données principales	6
1.6.2	Relations principales	6
1.6.3	Méthodes utiles	6
1.7	Gestion des missions	6
1.7.1	Lister les missions	7
1.7.2	Créer une mission	7
1.7.3	Afficher une mission	7
1.7.4	Modifier et supprimer une mission	7
1.7.5	Mes missions	7
1.7.6	Adaptation des noms de champs	8
1.8	Gestion des candidatures	8
1.8.1	Voir les candidatures	8
1.8.2	Envoyer une candidature	8
1.8.3	Accepter ou refuser une candidature	8
1.8.4	Finaliser une mission	9
1.9	Système de messagerie	9
1.9.1	Conversations	9
1.9.2	Lecture des messages	9
1.9.3	Envoi de message	9
1.9.4	Démarrage d'une conversation	10
1.10	Statistiques administrateur	10
1.11	Routes API du Backend	10
1.11.1	Routes publiques	10

1.11.2	Routes protégées	11
1.11.3	Routes admin	11
1.12	Base de données côté Backend	11
1.12.1	Table <code>users</code>	11
1.12.2	Table <code>missions</code>	11
1.12.3	Table <code>applications</code>	12
1.12.4	Tables de messagerie	12
2	Explication détaillée du Frontend	13
2.1	Rôle du Frontend	13
2.2	Technologies utilisées dans le Frontend	13
2.3	Organisation du Frontend	13
2.4	Observation sur l'architecture Frontend	14
2.5	Point d'entrée et configuration	14
2.6	Communication avec le Backend	14
2.7	Gestion de l'authentification côté Frontend	15
2.7.1	Fonctions du service	15
2.7.2	Gestion de session	15
2.7.3	Intercepteur d'authentification	15
2.7.4	Intercepteur d'erreur	15
2.8	Module d'authentification	15
2.8.1	Écran de connexion	16
2.8.2	Écran d'inscription	16
2.8.3	Mot de passe oublié	16
2.8.4	Vérification du token	16
2.9	Module Client	16
2.9.1	Dashboard client	16
2.9.2	Publication d'une mission	17
2.9.3	Mes missions	17
2.9.4	Candidatures reçues	17
2.10	Module Freelance	17
2.11	Module Missions	17
2.11.1	Liste des missions	18
2.11.2	Détail d'une mission	18
2.12	Module Candidature	18
2.13	Module Messagerie	18
2.14	Module Admin	19
2.15	Services principaux du Frontend	19
2.15.1	<code>auth.service.ts</code>	19
2.15.2	<code>mission.service.ts</code>	19
2.15.3	<code>application.service.ts</code>	19
2.15.4	<code>message.service.ts</code>	20
2.15.5	<code>stats.service.ts</code>	20
2.16	Points importants à noter dans le Frontend	20
3	Conclusion	21

Chapitre 1

Explication détaillée du Backend

1.1 Rôle du Backend

Le backend est responsable de toute la logique métier de l'application. Il gère :

- l'authentification des utilisateurs ;
- la gestion des profils ;
- la publication des missions ;
- la gestion des candidatures ;
- la messagerie entre utilisateurs ;
- les statistiques administrateur.

En pratique, le backend reçoit les requêtes du frontend, traite les données, interagit avec la base de données, puis renvoie des réponses JSON.

1.2 Technologies utilisées dans le Backend

Le backend repose principalement sur :

- PHP 8.3 ;
- Laravel 13 ;
- Laravel Sanctum ;
- MySQL.

Le fichier `backend/composer.json` montre que Laravel est la base du projet et que l'authentification utilise surtout Sanctum pour générer et vérifier les tokens.

1.3 Organisation du Backend

Le backend suit l'organisation standard d'un projet Laravel.

1.3.1 Dossiers importants

- `backend/app/Models` : contient les modèles de données ;
- `backend/app/Http/Controllers/Api` : contient les contrôleurs API ;

- `backend/app/Http/Middleware` : contient les middlewares ;
- `backend/database/migrations` : contient la structure de la base de données ;
- `backend/routes/api.php` : contient les routes de l'API ;
- `backend/resources/views/emails` : contient les templates d'emails.

Cette organisation rend le projet lisible et sépare bien la logique métier, les routes et les modèles.

1.4 Authentification et gestion des utilisateurs

La gestion des utilisateurs est centralisée dans le fichier :

`backend/app/Http/Controllers/Api/AuthController.php`

1.4.1 Inscription

La méthode `register()` permet de créer un compte utilisateur. Elle :

- vérifie les champs obligatoires comme `name`, `email`, `password` et `role` ;
- crée un nouvel utilisateur dans la table `users` ;
- génère un token d'authentification ;
- retourne les informations du compte et le token.

Les rôles possibles sont :

- `client` ;
- `freelance` ;
- `admin`.

1.4.2 Connexion

La méthode `login()` :

- vérifie l'existence de l'utilisateur ;
- compare le mot de passe fourni avec celui stocké en base ;
- vérifie que le compte est actif ;
- génère un nouveau token.

Le token reçu sera ensuite utilisé par le frontend pour accéder aux routes protégées.

1.4.3 Profil

Le backend permet aussi :

- de récupérer l'utilisateur connecté avec `me()` ;
- de modifier son profil avec `updateProfile()`.

Les informations modifiables incluent par exemple :

- le nom ;

- le téléphone ;
- la ville ;
- la bio ;
- l’avatar.

1.4.4 Déconnexion

La méthode `logout()` supprime le token courant afin de fermer la session.

1.5 Réinitialisation du mot de passe

Le backend prend aussi en charge le mot de passe oublié.

1.5.1 Demande de réinitialisation

La méthode `forgotPassword()` :

- vérifie l’email ;
- cherche l’utilisateur correspondant ;
- génère un token aléatoire ;
- stocke ce token dans la table `password_reset_tokens` ;
- envoie un email de réinitialisation.

1.5.2 Mail envoyé

Le mail est géré par :

`backend/app/Mail/ResetPasswordMail.php`

Cette classe :

- reçoit le token et l’email ;
- construit un lien vers le frontend ;
- utilise la vue `emails.reset-password`.

1.5.3 Définition du nouveau mot de passe

La méthode `resetPassword()` :

- vérifie le token ;
- retrouve l’utilisateur ;
- met à jour son mot de passe ;
- supprime le token de réinitialisation.

Ce mécanisme montre que le backend contrôle tout le processus de sécurité lié au changement de mot de passe.

1.6 Le modèle Utilisateur

Le modèle :

`backend/app/Models/User.php`

représente un utilisateur.

1.6.1 Données principales

Le modèle contient notamment :

- `name`;
- `email`;
- `password`;
- `role`;
- `phone`;
- `city`;
- `bio`;
- `avatar`;
- `is_active`.

1.6.2 Relations principales

Un utilisateur peut :

- publier plusieurs missions s'il est client ;
- envoyer plusieurs candidatures s'il est freelance ;
- participer à plusieurs conversations ;
- envoyer plusieurs messages.

1.6.3 Méthodes utiles

Le modèle contient aussi :

- `isAdmin()`;
- `isClient()`;
- `isFreelance()`.

Ces méthodes servent à simplifier les vérifications de rôle dans les contrôleurs.

1.7 Gestion des missions

La logique des missions est gérée dans :

`backend/app/Http/Controllers/Api/MissionController.php`

1.7.1 Lister les missions

La méthode `index()` permet :

- de récupérer les missions ;
- de charger les informations du client ;
- de compter les candidatures ;
- de filtrer par statut ;
- de rechercher par mot-clé ;
- de paginer les résultats.

Cette méthode est utile surtout pour les freelances qui veulent parcourir les annonces.

1.7.2 Créer une mission

La méthode `store()` permet à un client de publier une mission. Elle :

- valide les données ;
- enregistre la mission ;
- associe automatiquement la mission au client connecté ;
- fixe le statut à `ouverte`.

1.7.3 Afficher une mission

La méthode `show()` retourne les détails d'une mission ainsi que :

- son client ;
- ses candidatures ;
- les freelances ayant postulé.

1.7.4 Modifier et supprimer une mission

Les méthodes `update()` et `destroy()` vérifient d'abord si l'utilisateur est autorisé.

En général, seuls :

- le client propriétaire ;
- l'administrateur.

peuvent modifier ou supprimer une mission.

1.7.5 Mes missions

La méthode `myMissions()` retourne les missions du client actuellement connecté.

1.7.6 Adaptation des noms de champs

Le backend contient une logique intéressante pour s'adapter au frontend. Il traduit certains champs français en champs anglais :

- `titre` devient `title`;
- `date_echeance` devient `deadline`;
- `statut` devient `status`;
- `categorie` devient `skills`.

Cela permet au frontend d'envoyer des noms de champs plus naturels en français, tout en gardant une structure interne plus technique dans la base.

1.8 Gestion des candidatures

La logique des candidatures se trouve dans :

`backend/app/Http/Controllers/Api/ApplicationController.php`

1.8.1 Voir les candidatures

La méthode `index()` adapte le résultat selon le rôle :

- un client voit les candidatures reçues sur ses missions;
- un freelance voit les candidatures qu'il a envoyées.

1.8.2 Envoyer une candidature

La méthode `store()` :

- vérifie que la mission existe;
- vérifie que la mission est encore ouverte;
- empêche les doubles candidatures;
- crée la candidature avec le statut `en_attente`.

Après cela, le backend envoie automatiquement un message au client pour le prévenir.

1.8.3 Accepter ou refuser une candidature

La méthode `updateStatus()` permet :

- d'accepter une candidature;
- de la refuser.

Si la candidature est acceptée :

- la mission passe à `en_cours`;
- un message est envoyé au freelance.

Si elle est refusée :

- un message est aussi envoyé au freelance.

1.8.4 Finaliser une mission

La méthode `complete()` permet au freelance de signaler qu'une mission acceptée est terminée.

Dans ce cas :

- la mission passe à **fermée** ;
- un message est envoyé au client.

Cette partie est importante car elle relie la gestion des candidatures au cycle de vie complet d'une mission.

1.9 Système de messagerie

La messagerie est gérée dans :

`backend/app/Http/Controllers/Api/MessageController.php`

1.9.1 Conversations

La méthode `conversations()` retourne :

- les conversations de l'utilisateur ;
- les participants ;
- le dernier message ;
- le nombre de messages non lus.

1.9.2 Lecture des messages

La méthode `messages()` :

- vérifie que l'utilisateur appartient à la conversation ;
- charge les messages ;
- marque comme lus les messages reçus.

1.9.3 Envoi de message

La méthode `sendMessage()` :

- contrôle l'accès à la conversation ;
- vérifie le contenu ;
- crée le message ;
- met à jour la date de dernière activité de la conversation.

1.9.4 Démarrage d'une conversation

La méthode `startConversation()` permet de commencer une conversation avec un autre utilisateur.

Si une conversation existe déjà entre les deux personnes, elle est réutilisée.

Cette messagerie est aussi utilisée automatiquement par le système lors :

- d'une nouvelle candidature ;
- d'une candidature acceptée ;
- d'une candidature refusée ;
- d'une mission terminée.

1.10 Statistiques administrateur

Le contrôleur :

`backend/app/Http/Controllers/Api/StatsController.php`

est réservé à l'administrateur.

Il fournit :

- le nombre total d'utilisateurs ;
- le nombre de clients et de freelances ;
- les nouveaux utilisateurs du mois ;
- le nombre total de missions ;
- le nombre de missions ouvertes, en cours et fermées ;
- le nombre total de candidatures ;
- les candidatures en attente, acceptées et refusées.

Il fournit aussi :

- l'activité des derniers mois ;
- la liste des utilisateurs récents.

Cela permet de construire un dashboard d'administration dans le frontend.

1.11 Routes API du Backend

Le fichier :

`backend/routes/api.php`

centralise les routes.

1.11.1 Routes publiques

- POST `/register` ;
- POST `/login` ;
- POST `/forgot-password` ;
- POST `/reset-password`.

1.11.2 Routes protégées

Après authentification :

- POST /logout;
- GET /me;
- PUT /profile;
- GET /missions;
- POST /missions;
- GET /applications;
- POST /applications;
- GET /conversations;
- POST /conversations.

1.11.3 Routes admin

Pour l'administrateur :

- GET /stats;
- GET /stats/dashboard;
- GET /stats/monthly;
- GET /stats/users.

1.12 Base de données côté Backend

Les migrations montrent les principales tables du projet.

1.12.1 Table users

Elle contient les données des utilisateurs et leur rôle.

1.12.2 Table missions

Elle contient :

- le titre;
- la description;
- le budget;
- le statut;
- les compétences;
- la date limite;
- l'identifiant du client.

1.12.3 Table applications

Elle contient :

- la mission concernée ;
- le freelance ;
- le statut de candidature ;
- un message ;
- un budget proposé.

1.12.4 Tables de messagerie

Le système de discussion repose sur :

- `conversations` ;
- `conversation_participants` ;
- `messages`.

Ces tables permettent une messagerie privée simple entre utilisateurs.

Chapitre 2

Explication détaillée du Frontend

2.1 Rôle du Frontend

Le frontend est l'interface visible par l'utilisateur. Il permet d'interagir avec l'application sans accéder directement à la base de données ou à la logique Laravel.

Son rôle est de :

- afficher les formulaires ;
- envoyer les requêtes au backend ;
- présenter les missions ;
- présenter les candidatures ;
- afficher la messagerie ;
- gérer l'expérience utilisateur.

2.2 Technologies utilisées dans le Frontend

Le frontend utilise :

- Angular 19 ;
- TypeScript ;
- RxJS ;
- Angular Router ;
- HttpClient.

Le fichier `frontend/package.json` montre les dépendances principales du projet.

2.3 Organisation du Frontend

Le frontend est situé dans :

`frontend/src/app`

On y trouve plusieurs grandes zones :

- `core` : services, constantes, guards, intercepteurs ;

- `modules` : grands modules fonctionnels ;
- `auth` et `client` : une autre organisation plus récente ou parallèle ;
- `shared` : composants réutilisables.

Cette structure montre que le projet frontend a évolué avec le temps.

2.4 Observation sur l’architecture Frontend

Le code frontend contient deux styles qui coexistent :

- une architecture classique avec `AppModule` et `AppRoutingModule` ;
- une architecture plus moderne avec composants standalone et `app.routes.ts`.

Cela signifie probablement que le frontend est en cours de migration ou qu’il contient deux versions de développement.

Malgré cela, les fonctionnalités principales sont bien identifiables.

2.5 Point d’entrée et configuration

Le point d’entrée du frontend est :

```
frontend/src/main.ts
```

Ce fichier démarre `AppModule`, ce qui indique que l’application utilise principalement l’architecture modulaire classique.

Le routage principal se trouve dans :

```
frontend/src/app/app-routing-module.ts
```

Les principales sections chargées sont :

- `auth` ;
- `client` ;
- `freelance` ;
- `admin` ;
- `missions` ;
- `candidature` ;
- `messagerie` ;
- `profile`.

2.6 Communication avec le Backend

Le frontend communique avec le backend via HTTP.

Le fichier :

```
frontend/src/app/core/constants/api.constants.ts
```

définit l’URL de base de l’API. Il construit :

- `API_BASE_URL` ;
- `API_URL`.

Le frontend envoie ensuite ses requêtes vers les endpoints du backend Laravel.

2.7 Gestion de l'authentification côté Frontend

Le service principal est :

```
frontend/src/app/core/services/auth.service.ts
```

2.7.1 Fonctions du service

Il permet :

- de se connecter ;
- de s'inscrire ;
- de se déconnecter ;
- de récupérer l'utilisateur courant ;
- de connaître le rôle courant ;
- d'appeler l'endpoint `/me`.

2.7.2 Gestion de session

Le frontend stocke dans `localStorage` :

- le token ;
- l'utilisateur.

Cela permet de maintenir la session entre les rechargements de page.

2.7.3 Intercepteur d'authentification

Le fichier :

```
frontend/src/app/core/interceptors/auth.interceptor.ts
```

ajoute automatiquement le token dans les en-têtes HTTP.

2.7.4 Intercepteur d'erreur

Le fichier :

```
frontend/src/app/core/interceptors/error.interceptor.ts
```

gère les erreurs 401 et supprime les données de session si l'utilisateur n'est plus autorisé.

2.8 Module d'authentification

Le module :

```
frontend/src/app/modules/auth
```

contient les écrans liés à l'accès utilisateur.

2.8.1 Écran de connexion

Le composant `login` :

- lit l'email et le mot de passe ;
- appelle le service d'authentification ;
- redirige l'utilisateur selon son rôle.

2.8.2 Écran d'inscription

Le composant `register` :

- récupère les informations du nouvel utilisateur ;
- valide la confirmation du mot de passe ;
- appelle l'API d'inscription.

2.8.3 Mot de passe oublié

Le composant `forgot-password` :

- demande l'email ;
- appelle l'API de demande de réinitialisation ;
- affiche un message de retour.

2.8.4 Vérification du token

Le composant `verify-code` :

- lit le token dans l'URL ;
- vérifie le nouveau mot de passe ;
- appelle l'API de réinitialisation.

Ce module est donc directement connecté au backend d'authentification.

2.9 Module Client

Le frontend contient des écrans dédiés au rôle client.

2.9.1 Dashboard client

Le dashboard client affiche en général :

- les informations de l'utilisateur ;
- des statistiques simples ;
- le nombre de missions créées ;
- le nombre de missions ouvertes.

2.9.2 Publication d'une mission

Le composant `publier-mission` permet au client :

- de saisir le titre ;
- de saisir la description ;
- de choisir une catégorie ;
- de définir un budget ;
- de fixer une date d'échéance.

Le formulaire envoie les données au backend.

2.9.3 Mes missions

Le composant `mes-missions` :

- charge les missions du client ;
- les affiche ;
- permet leur suppression.

2.9.4 Candidatures reçues

Le composant `candidatures-recues` permet :

- de charger les missions du client ;
- d'afficher les candidatures reçues pour une mission.

Cette partie correspond à la gestion quotidienne des annonces côté client.

2.10 Module Freelance

Le module `freelance` contient le tableau de bord du freelance.

Le composant principal affiche :

- les missions disponibles ;
- les candidatures déjà envoyées ;
- des statistiques rapides.

Il permet au freelance d'avoir une vue claire sur son activité.

2.11 Module Missions

Le module `missions` sert à explorer les offres publiées.

2.11.1 Liste des missions

Le composant `mission-list` :

- charge les missions ;
- filtre par statut ;
- filtre par texte ;
- permet de consulter les détails ;
- permet de postuler.

2.11.2 Détail d’une mission

Le composant `mission-detail` :

- affiche les informations complètes d’une mission ;
- montre le client associé ;
- permet d’envoyer une candidature ;
- permet de contacter le client.

Ce module relie donc consultation, candidature et messagerie.

2.12 Module Candidature

Le composant `candidature-tracker` est une partie importante du frontend.

Il permet :

- de suivre les candidatures envoyées ;
- de filtrer par statut ;
- d’accepter ou refuser une candidature côté client ;
- de marquer une mission comme terminée côté freelance.

Cette interface correspond au suivi de l’état d’avancement des relations client-freelance.

2.13 Module Messagerie

Le module de messagerie repose principalement sur :

`messagerie/chat`

Il permet :

- d’afficher les conversations ;
- de charger les messages ;
- d’envoyer un message ;
- de démarrer une nouvelle conversation.

Le frontend charge aussi la liste des autres utilisateurs afin de permettre la création d’une conversation manuelle.

Le composant gère également :

- le scroll automatique ;
- la sélection d’une conversation ;
- l’identification de l’autre participant.

2.14 Module Admin

Le frontend contient aussi un tableau de bord administrateur.

Ce dashboard utilise `StatsService` pour récupérer :

- le nombre d'utilisateurs ;
- le nombre de missions ;
- le nombre de candidatures ;
- les missions terminées ;
- l'activité mensuelle ;
- les utilisateurs récents.

Cette partie transforme les statistiques du backend en informations visuelles pour l'administrateur.

2.15 Services principaux du Frontend

Les services du dossier :

```
frontend/src/app/core/services
```

sont centraux.

2.15.1 `auth.service.ts`

Il gère l'authentification et la session utilisateur.

2.15.2 `mission.service.ts`

Il gère :

- le chargement des missions ;
- la création ;
- la mise à jour ;
- la suppression.

2.15.3 `application.service.ts`

Il gère :

- l'envoi des candidatures ;
- le chargement des candidatures ;
- le changement de statut ;
- la finalisation.

2.15.4 `message.service.ts`

Il gère :

- les conversations ;
- les messages ;
- les nouveaux échanges.

2.15.5 `stats.service.ts`

Il gère les appels aux endpoints d'administration.

2.16 Points importants à noter dans le Frontend

En lisant le code, on remarque plusieurs éléments importants :

- le frontend est fonctionnel mais pas totalement unifié ;
- il existe des doublons de services et de guards ;
- il existe plusieurs styles de routage ;
- certaines parties semblent anciennes et d'autres plus récentes.

Malgré cela, la logique métier générale reste claire :

- connexion ;
- publication de mission ;
- candidature ;
- suivi ;
- messagerie ;
- administration.

Chapitre 3

Conclusion

Le backend et le frontend jouent des rôles complémentaires dans ce projet.

Le backend Laravel :

- centralise la logique métier ;
- gère les utilisateurs ;
- sécurise l'accès ;
- pilote les missions, candidatures et messages ;
- fournit les statistiques.

Le frontend Angular :

- affiche les interfaces utilisateur ;
- gère la navigation ;
- envoie les requêtes vers l'API ;
- permet l'interaction avec toutes les fonctionnalités métier.

En résumé, le backend est le cerveau de l'application et le frontend est la partie visible qui permet à l'utilisateur d'exploiter toutes les fonctionnalités de la plateforme.